



---

# HeeraMon

## Technical Architecture Specification

### Architecture Specification Document

---

**Date of Preparation:** September 30, 2026

|                   |                                              |
|-------------------|----------------------------------------------|
| Document ID       | HM-DOC-07                                    |
| Title             | Technical Architecture Specification         |
| Classification    | Architecture Specification Document          |
| Status            | Draft                                        |
| Version           | 1.0                                          |
| Owner             | Product / Engineering                        |
| Audience          | All project contributors                     |
| Related Documents | SRS (HM-DOC-02), AI / Agent Spec (HM-DOC-04) |

# Contents

|                                            |          |
|--------------------------------------------|----------|
| <b>1 Purpose</b>                           | <b>2</b> |
| <b>2 Architectural Principle</b>           | <b>2</b> |
| <b>3 High-Level Components</b>             | <b>3</b> |
| 3.1 Frontend                               | 3        |
| 3.2 Backend API                            | 3        |
| 3.3 Agent Layer                            | 3        |
| 3.4 Job Ingestion Layer                    | 4        |
| 3.5 Matching Layer                         | 4        |
| 3.6 Background Worker / Scheduler          | 4        |
| 3.7 Database                               | 4        |
| 3.8 External Services                      | 5        |
| <b>4 Proposed Stack</b>                    | <b>5</b> |
| 4.1 Frontend                               | 5        |
| 4.2 Backend                                | 5        |
| 4.3 Database / Authentication              | 5        |
| 4.4 Agent Orchestration                    | 5        |
| 4.5 LLM                                    | 5        |
| <b>5 Agent Architecture</b>                | <b>6</b> |
| <b>6 LLM Provider Abstraction</b>          | <b>6</b> |
| <b>7 Data and State</b>                    | <b>6</b> |
| <b>8 Background Execution</b>              | <b>7</b> |
| <b>9 Reliability and Failure Isolation</b> | <b>7</b> |
| <b>10 Security Boundaries</b>              | <b>8</b> |
| <b>11 Cost Strategy</b>                    | <b>8</b> |
| <b>12 Architectural Boundaries</b>         | <b>8</b> |
| <b>13 Success Criteria</b>                 | <b>9</b> |
| <b>14 Document Control</b>                 | <b>9</b> |

## 1 Purpose

This document defines the technical architecture of HeeraMon.

The architecture should be:

- production-oriented from the start;
- simple enough for a small team or individual developer;
- inexpensive to operate initially;
- testable and observable;
- replaceable and extensible;
- suitable for future background agents and larger job volume.

The architecture should avoid unnecessary complexity without creating a dead end for future versions.

## 2 Architectural Principle

HeeraMon follows an:

**LLM-assisted, not LLM-dependent**

architecture.

The LLM should handle tasks where language understanding and reasoning provide real value, such as:

- interpreting user messages;
- extracting preferences;
- understanding job descriptions;
- resolving ambiguity;
- explaining trade-offs;
- deciding when a tool is useful.

Deterministic software should handle tasks such as:

- authentication;
- database operations;
- filtering;
- deduplication;
- scheduling;
- notifications;
- state management;
- security;

- rate limiting;
- validation.

This reduces cost, improves reliability, and makes the system easier to test.

## 3 High-Level Components

HeeraMon consists of the following major components:

### 3.1 Frontend

Responsible for:

- conversational interface;
- onboarding;
- job presentation;
- preference visibility and editing;
- notifications;
- application review;
- authentication.

### 3.2 Backend API

Responsible for:

- business logic;
- authentication/session handling;
- user and job data access;
- orchestration between frontend, agent, and services;
- API endpoints;
- security and validation.

### 3.3 Agent Layer

Responsible for:

- conversation reasoning;
- preference extraction;
- job interpretation;
- trade-off reasoning;
- tool selection;
- conversational decisions.

The system uses one agent with multiple tools, rather than a multi-agent architecture.

### 3.4 Job Ingestion Layer

Responsible for:

- source adapters;
- fetching jobs;
- normalization;
- validation;
- deduplication;
- freshness tracking.

### 3.5 Matching Layer

Responsible for:

- deterministic constraint filtering;
- preference comparison;
- candidate selection;
- structured match information.

The LLM may interpret or explain results, but should not be required for every basic filtering operation.

### 3.6 Background Worker / Scheduler

Responsible for:

- periodic job searches;
- source ingestion;
- matching new jobs;
- notification triggering;
- maintenance tasks.

### 3.7 Database

Responsible for persistent application state including:

- users;
- preferences;
- conversations;
- jobs;
- matches;
- applications;
- notifications;

- agent state;
- source metadata.

### 3.8 External Services

May include:

- LLM providers;
- job sources;
- authentication providers;
- email/push notification services;
- external application systems.

External dependencies should be isolated behind service interfaces where practical.

## 4 Proposed Stack

The initial implementation may use:

### 4.1 Frontend

- Next.js
- TypeScript
- Tailwind CSS
- Framer Motion

### 4.2 Backend

- Python
- FastAPI

### 4.3 Database / Authentication

- PostgreSQL
- Supabase

### 4.4 Agent Orchestration

- LangGraph

### 4.5 LLM

- Provider abstraction supporting hosted or compatible models

The exact hosting provider may remain flexible during implementation.

These choices are implementation defaults, not permanent product requirements.

## 5 Agent Architecture

HeeraMon should use:

**One agent + multiple deterministic tools**

Example tools may include:

- `get_user_preferences`
- `update_user_preferences`
- `search_jobs`
- `get_job_details`
- `compare_job_to_preferences`
- `get_user_profile`
- `get_cv`
- `prepare_application`
- `save_decision`
- `send_notification`

The agent should call tools when necessary rather than directly manipulating application state through uncontrolled model output.

## 6 LLM Provider Abstraction

LLM access should be isolated behind a provider layer.

Conceptually:

**HeeraMon Agent → LLM Provider Interface → Selected Model**

The provider may later support:

- HeeraMon-hosted models/API access;
- OpenAI;
- Google;
- Anthropic;
- compatible local/open models;
- user-provided API keys where appropriate.

The rest of the application should not depend directly on one model provider.

## 7 Data and State

PostgreSQL should be the primary source of truth for persistent application data.

Agent state should be persisted where required for:

- continuing conversations;
- background execution;
- interrupted workflows;
- human approval steps;
- application preparation;
- future agent resumption.

Transient model output should not be treated as authoritative application state.

## 8 Background Execution

Continuous job seeking must not depend on the user keeping the browser open.

Background processes should be able to:

1. retrieve new jobs;
2. normalize and store them;
3. evaluate relevant users;
4. identify meaningful matches;
5. create notifications;
6. allow the user to resume the conversation later.

Background jobs should be idempotent where practical so that retries do not create duplicate records or actions.

## 9 Reliability and Failure Isolation

External failures should not bring down the core application.

Examples:

- LLM unavailable → conversational functions may degrade, while stored data remains accessible.
- Job source unavailable → other sources continue operating.
- Notification failure → notification can be retried.
- Application site unavailable → application remains pending rather than being falsely marked successful.

Important operations should have:

- timeouts;
- retries where appropriate;

- error logging;
- clear failure states.

## 10 Security Boundaries

Security-sensitive operations should remain under deterministic backend control.

The LLM must not independently control:

- authentication;
- authorization;
- secrets;
- unrestricted database access;
- arbitrary code execution;
- final application submission.

Tool permissions should follow least privilege.

User approval must be enforced by application logic, not merely by an instruction in the system prompt.

## 11 Cost Strategy

The system should target minimal or zero infrastructure cost during development and early experimentation where practical.

Cost should be controlled through:

- deterministic filtering before LLM calls;
- limited background search frequency;
- caching;
- deduplication;
- concise prompts;
- model/provider flexibility;
- free-tier infrastructure where appropriate;
- avoiding unnecessary agent calls.

The architecture should allow paid infrastructure and higher-capacity models to be introduced later without major redesign.

## 12 Architectural Boundaries

The initial release does not require:

- microservices;
- Kubernetes;
- multiple autonomous agents;
- complex event-driven infrastructure;
- dedicated vector databases;
- a large data warehouse;
- advanced distributed systems;
- custom ML ranking infrastructure.

A modular application with clear service boundaries is preferred over premature distributed architecture.

## 13 Success Criteria

The architecture is successful when it can support the complete core loop:

**User → Conversation → Preferences → Job Ingestion → Matching → Agent Reasoning → Notification → User Decision → Application Preparation → Human Approval**

while remaining:

- understandable;
- testable;
- secure;
- affordable;
- observable;
- replaceable where external providers are concerned;
- extensible toward future versions.

**Core principle:** Build the system as a small production system, not as a disposable prototype.

## 14 Document Control

| Version | Date               | Author / Role         | Changes         |
|---------|--------------------|-----------------------|-----------------|
| 1.0     | September 30, 2026 | Product / Engineering | Initial release |

**Approval:** This document defines the technical architecture for HeeraMon. All subsequent design and implementation work must remain consistent with the principles stated herein.

*HeeraMon flies to help people find job that fits not only their career, but the life they want to live.*



---

**Saadman Sakib**

Product Engineer